



Administration

▶ Security

▶ Self-Managed Deployments

▶ FAQ

▼ Reference

▶ Collation

▶ Connection Strings

▶ Database Commands

DDL Operations

Default Port

Read & Write Concerns

Error Codes

Glossary

Log Messages

Limits & Thresholds

MongoDB Database Tools

▶ Wire Protocol

▶ mongosh Methods

▼ Operators

▼ Query & Projection

▶ Comparison Query

▶ Logical Query

▶ Element Query

▼ Evaluation Query

\$expr

▼ DOCS MENU



✦ Ask MongoDB AI



Docs Home / Data / Re / Op / Que / Evaluation Query

\$regex

On this page

Definition

Compatibility

Syntax

Behavior

Examples

NOTE

This page describes regular expression search capabilities for self-managed (non-Atlas) deployments. For data hosted on MongoDB Atlas, MongoDB offers an improved full-text search solution, **Atlas Search**, which has its own `$regex` operator. To learn more, see `$regex` in the Atlas Search documentation.

Definition

`$regex`

Provides regular expression capabilities for pattern matching *strings* in queries.

Compatibility

You can use `$regex` for deployments hosted in the following environments:

- **MongoDB Atlas:** The fully managed service **for MongoDB deployments** in the cloud
- **MongoDB Enterprise:** The subscription-based, self-managed version of MongoDB
- **MongoDB Community:** The source-available, free-to-use, and self-managed version of MongoDB

Syntax

To use `$regex`, use one of the following syntaxes:

```
{ <field>: { $regex: /pattern/, $op  
{ "<field>": { "$regex": "pattern",  
{ <field>: { $regex: /pattern/<opti
```

NOTE

To use `$regex` with `mongodump`, you must enclose the query document in single quotes (`'{ ... }'`) to ensure that it does not interact with your shell environment.

The query document must be in **Extended JSON v2** format (either relaxed or ~~canonical/strict mode~~), which includes enclosing the field names and operators in quotes. For example:

```
mongodump -d=sample_mflix -c=movie
```

In MongoDB, you can also use regular expression objects (i.e. `/pattern/`) to specify regular expressions:

```
{ <field>: /pattern/<options> }
```



For restrictions on particular syntax use, see [\\$regex vs. /pattern/ Syntax](#).

\$options

The following `<options>` are available for use with regular expression.

Option	Description
<code>i</code>	Case insensitivity to match upper and lower cases. For an example, see Perform Case-Insensitive Regular Expression Match .
<code>m</code>	For patterns that include anchors (i.e. <code>^</code> for the start, <code>\$</code> for the end), match at the beginning or end of each line for strings with multiline values. Without this option, these anchors match at beginning or end of the string. For an example, see Multiline Match for Lines Starting with Specified Pattern. If the pattern contains no anchors or if the string value has no newline characters (e.g. <code>\n</code>), the <code>m</code> option has no effect.
<code>x</code>	"Extended" capability to ignore all white space characters in the <code>\$regex</code> pattern unless escaped or included in a character class. Additionally, it ignores characters in-between and including an un-escaped hash/pound (<code>#</code>) character and the next new line, so that you may include comments in complicated patterns. This only applies to data characters; white space characters may never appear within special character sequences in a pattern. The <code>x</code> option does not affect the handling of the VT character (i.e. code 11).

Option	Description
<code>s</code>	Allows the dot character (i.e. <code>.</code>) to match all characters <i>including</i> newline characters. For an example, see Use the <code>.</code> Dot Character to Match New Line .
<code>u</code>	Supports Unicode. This flag is accepted, but is redundant. UTF is set by default in the <code>\$regex</code> operator, making the <code>u</code> option unnecessary.

NOTE

The `$regex` operator *does not* support the global search modifier `g`.

Behavior

`$regex` vs. `/pattern/` Syntax

`$in` Expressions

To include a regular expression in an `$in` query predicate operator, you can only use JavaScript regular expression objects (`/pattern/`).

For example:

```
{ name: { $in: [ /^acme/i, /^ack/ ] }
```

You *cannot* use `$regex` operator expressions inside an `$in` operator.

Implicit `AND` Conditions for the Field

To include a regular expression in a comma-separated list of query conditions for the field, use the `$regex` operator. For example:

```
{ name: { $regex: /acme.*corp/i, $options: 'i' } }
{ name: { $regex: /acme.*corp/, $options: 's' } }
{ name: { $regex: 'acme.*corp', $options: 'si' } }
```

`x` and `s` Options

To use either the `x` option or `s` options, you must use the `$regex` operator expression *with* the `$options` operator. For example, to specify the `i` and the `s` options, you must use `$options` for both:

```
{ name: { $regex: /acme.*corp/, $options: 'si' } }
{ name: { $regex: 'acme.*corp', $options: 'si' } }
```

PCRE Versus JavaScript

To use PCRE-supported features in a regular expression that aren't supported in JavaScript, you must use the `$regex` operator and specify the regular expression as a string.

To match case-insensitive strings:

- `"(?i)"` begins a case-insensitive match.
- `"(?-i)"` ends a case-insensitive match.

For example, the regular expression `"(?i)a(?-i)cme"` matches strings that:

- Begin with `"a"` or `"A"`. This is a case-insensitive match.
- End with `"cme"`. This is a case-sensitive match.

These strings match the example regular expression:

- "acme"
- "Acme"

The following example uses the `$regex` operator to find `name` field strings that match the regular expression `"(?i)a(?-i)cme"`:

```
{ name: { $regex: "(?i)a(?-i)cme" } }
```

Starting in version 6.1, MongoDB uses the PCRE2 (Perl Compatible Regular Expressions) library to implement regular expression pattern matching. To learn more about PCRE2, see the [PCRE Documentation](#).[↗]

`$regex` and `$not`

The `$not` operator can perform logical NOT operation on both:

- Regular expression objects (i.e. `/pattern/`)

For example:

```
db.inventory.find( { item: { $r
```

- `$regex` operator expressions

For example:

```
db.inventory.find( { item: { $r  
db.inventory.find( { item: { $r
```

Index Use

Index use and performance for `$regex` queries varies depending on whether the query is case-sensitive or case-insensitive.

Case-Sensitive Queries

For case sensitive regular expression queries, if an index exists for the field, then MongoDB matches the regular expression against the values in the index, which can be faster than a collection scan.

Further optimization can occur if the regular expression is a "prefix expression", which means that all potential matches start with the same string. This allows MongoDB to construct a "range" from that prefix and only match against those values from the index that fall within that range.

A regular expression is a "prefix expression" if it starts with a caret (^) or a left anchor (\A), followed by a string of simple symbols. For example, the regex `/^abc.*/` will be optimized by matching only against the values from the index that start with `abc`.

Additionally, while `/^a/`, `/^a.*/`, and `/^a.*$/` match equivalent strings, they have different performance characteristics. All of these expressions use an index if an appropriate index exists; however, `/^a.*/`, and `/^a.*$/` are slower. `/^a/` can stop scanning after matching the prefix.

Case-Insensitive Queries

Case-insensitive indexes typically do not improve performance for `$regex` queries. The `$regex` implementation is not collation-aware

and cannot utilize case-insensitive indexes efficiently.

Examples

The examples in this section use the following `products` collection:

```
db.products.insertMany( [  
  { _id: 100, sku: "abc123", descr  
  { _id: 101, sku: "abc789", descr  
  { _id: 102, sku: "xyz456", descr  
  { _id: 103, sku: "xyz789", descr  
  { _id: 104, sku: "Abc789", descr  
] )
```



Perform a **LIKE** Match

The following example matches all documents where the `sku` field is like `"%789"`:

```
db.products.find( { sku: { $regex:
```



The example is analogous to the following SQL LIKE statement:

```
SELECT * FROM products  
WHERE sku like "%789";
```



Example output:

```
[  
  { _id: 101, sku: 'abc789', descripti  
  { _id: 103, sku: 'xyz789', descripti  
  { _id: 104, sku: 'Abc789', descripti  
]
```


Perform Case-Insensitive Regular Expression Match

The following example uses the `i` option to perform a *case-insensitive* match for documents with `sku` value that starts with `ABC`.

```
db.products.find( { sku: { $regex:
```



Example output:

```
[
  { _id: 100, sku: 'abc123', descripti
  { _id: 101, sku: 'abc789', descripti
  { _id: 104, sku: 'Abc789', descripti
]
```

Multiline Match for Lines Starting with Specified Pattern

The following example uses the `m` option to match lines starting with the letter `S` for multiline strings:

```
db.products.find( { description: {
```



Example output:

```
[
  { _id: 100, sku: 'abc123', descripti
  { _id: 101, sku: 'abc789', descripti
  { _id: 104, sku: 'Abc789', descripti
]
```

Without the `m` option, the example output is:

```
[
  { _id: 100, sku: 'abc123', descripti
  { _id: 104, sku: 'Abc789', descripti
]
```

If the `$regex` pattern does not contain an anchor, the pattern matches against the string as a whole, as in the following example:

```
db.products.find( { description: { 
```

Example output:

```
[
  { _id: 100, sku: 'abc123', descripti
  { _id: 101, sku: 'abc789', descripti
  { _id: 104, sku: 'Abc789', descripti
]
```

Use the `.` Dot Character to Match New Line

The following example uses the `s` option to allow the dot character (i.e. `.`) to match all characters *including* new line as well as the `i` option to perform a case-insensitive match:

```
db.products.find( { description: { 
```

Example output:

```
[
  { _id: 102, sku: 'xyz456', descripti
  { _id: 103, sku: 'xyz789', descripti
]
```

Without the `s` option, the example output is:

```
[
  { _id: 102, sku: 'xyz456', descripti
]
```

Ignore White Spaces in Pattern

The following example uses the `x` option ignore white spaces and the comments, denoted by the `#` and ending with the `\n` in the matching pattern:

```
var pattern = "abc #category code\n"
db.products.find( { sku: { $regex:
```



Example output:

```
[
  { _id: 100, sku: 'abc123', descripti
]
```

Use a Regular Expression to Match Case in Strings

The following example uses the regular expression `"(?i)a(?-i)bc"` to match `sku` field strings that contain:

- `"abc"`
- `"Abc"`

```
db.products.find( { sku: { $regex:
```



Example output:

```
[
  { _id: 100, sku: 'abc123', descripti
  { _id: 101, sku: 'abc789', descripti
  { _id: 104, sku: 'Abc789', descripti
]
```

Extend Regex Options to Match Characters Outside of ASCII

New in version 6.1.

By default, certain regex options (such as `/b` and `/w`) only recognize ASCII characters. This can cause unexpected results when performing regex matches against UTF-8 characters.

Starting in MongoDB 6.1, you can specify the `*UCP` regex option to match UTF-8 characters.

IMPORTANT

Performance of UCP Option

The `*UCP` option results in slower queries than those without the option specified because `*UCP` requires a multistage table lookup to perform the match.

For example, consider the following documents in a `songs` collection:

```
db.songs.insertMany( [
  { _id: 0, "artist" : "Blue Öyste
  { _id: 1, "artist": "Blue Öyster
  { _id: 2, "artist" : "Blue Oyste
] )
```



The following regex query uses the `\b` option in a regex match. The `\b` option matches a word boundary.

```
db.songs.find( { artist: { $regex:
```



Example output:

```
[
  { _id: 0, artist: 'Blue Öyster Cult'
  { _id: 1, artist: 'Blue Öyster Cult'
]
```

The previous results are unexpected because none of the words in the returned `artist` fields begin with the matched string (`yster`). The `Ö` character in documents `_id: 0` and `_id: 1` is ignored when performing the match because it is a UTF-8 character.

The expected result is that the query does not return any documents.

To allow the query to recognize UTF-8 characters, specify the `*UCP` option before the pattern:

```
db.songs.find( { artist: { $regex:
```



The previous query does not return any documents, which is the expected result.

TIP

Escape Characters for Regex Patterns

When specifying `*UCP` or any other regular expression option, ensure that you use the correct escape characters for your shell or driver.

 English

About

- Careers
- Investor Relations
- Legal
- GitHub
- Security Information
- Trust Center
- Connect with Us

Support

- Contact Us
- Customer Portal
- Atlas Status
- Customer Support
- Manage Cookies

Deployment Options

- MongoDB Atlas
- Enterprise Advanced

Community Edition

© 2024 MongoDB, Inc.